sentinel.

SÉCURITÉ DES SYSTÈMES DISTRIBUÉS

Authentification et autorisation avec Spring Security, JWT et React.

Conception et réalisation d’une application de suivi des incidents informatiques et de cybersécurité.

Devoir d’application · 4 novembre 2026

Spring Boot 3.5.16 · Java 21 · React 19 · JPA · H2

Auteur : Ahmed Douyry

Établissement : ENSET Mohammedia

01. Objectif et périmètre

Sentinel permet aux employés de signaler un incident, aux techniciens de traiter leurs affectations et aux administrateurs d'organiser la prise en charge. L'interface est en français et s'adapte aux écrans de bureau, tablette et mobile.

Rôle	Fonctionnalités
EMPLOYEE	Inscription, profil, création et consultation de ses incidents, tableau de bord personnel.
TECHNICIAN	Incidents affectés, changement de statut, notes de traitement, statistiques personnelles.
ADMIN	Tous les incidents, affectation, utilisateurs, activation et désactivation, statistiques globales.

Fonctions transversales

- Authentification JWT, BCrypt, session stateless et autorisations côté serveur.
- Validation des données et réponses JSON explicites pour les erreurs.
- Historique persistant des événements et notes de traitement.
- Modification du profil et du mot de passe avec révocation des sessions.
- Refresh token optionnel avec rotation et stockage sous forme de hash.

Choix de réalisation

Les projets initialisés ont été conservés. Le backend utilise la branche Spring Boot 3.5 disponible dans l'environnement et Java 21. Le frontend garde React 19, Vite et TypeScript : les composants portent l'extension `.tsx`, équivalente typée aux `.jsx` proposés dans le sujet. H2 fournit une base locale persistante sans installation d'un serveur MySQL.

> La sécurité des données est appliquée dans les services backend. Masquer une action ou protéger une route React sert à guider l'utilisateur ; une requête HTTP directe reste soumise aux mêmes contrôles de rôle et de propriété.

Organisation du projet

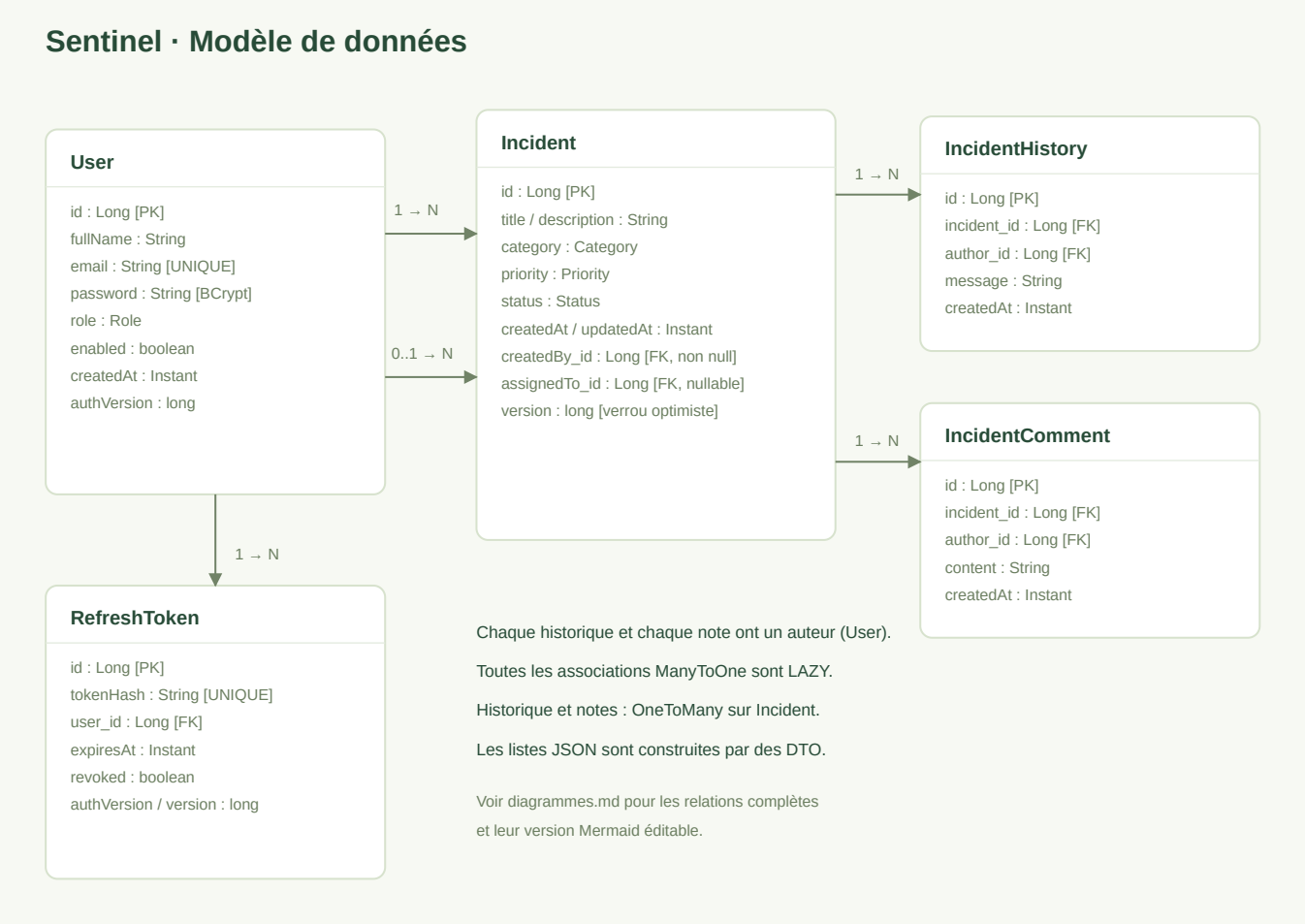
`backend/src/main/java/org/example/backend` est divisé en domain/entity, domain/enums, repository, dto/request, dto/response, mapper, service, security, config, bootstrap et web/controller/advice. Les contrôleurs délèguent sans accès JPA ; les services portent transactions et autorisations. Les mappers isolent le contrat JSON des entités. `frontend/src` distingue composants, contexte d'authentification, pages et services Axios. `docs` rassemble les documents et la collection Postman ; `output` contient les preuves de vérification.

02. Architecture générale



Le navigateur communique avec l'API REST en HTTP/JSON. Axios ajoute le jeton Bearer. Le filtre JWT établit l'authentification, puis les controllers délèguent aux services transactionnels. JPA assure la persistance et la cohérence des modifications.

03. Modèle de données et relations JPA



Justification des relations

Un incident a un auteur obligatoire et un technicien facultatif : deux associations `@ManyToOne(fetch = LAZY)` vers User. L'historique et les notes forment deux collections `@OneToMany(mappedBy = "incident")`, ordonnées et en cascade sur l'incident. User n'expose pas de collections inverses inutiles : les repositories filtrent les incidents par identifiant.

ManyToOne charge normalement la relation en EAGER si aucun mode n'est précisé ; LAZY diffère l'initialisation. Les DTO sont construits dans la transaction. Les champs des entités sont privés ; leurs accesseurs initialisent les proxys au besoin, et le hash du mot de passe ne fait jamais partie des réponses. Les listes ne chargent pas l'historique et les commentaires ; @EntityGraph récupère l'auteur et le technicien sans requête par ligne. Les statistiques utilisent des compteurs en base. Pour un grand volume, des projections et la pagination réduiraient le coût des listes.

04. Authentification et JWT

Inscription et connexion

`POST /auth/register` valide le nom, l'email et le mot de passe. L'email est normalisé, son unicité vérifiée, puis le mot de passe encodé par BCrypt. Le DTO n'accepte pas le rôle : le service impose EMPLOYEE. La connexion refuse les identifiants incorrects et les comptes désactivés.

Claims et signature

Un claim est une information contenue dans le payload du JWT. Le token signé avec HMAC contient `sub` (email), `uid`, `role`, `iat` (création), `exp` (expiration) et `version`. Le payload est encodé, sans chiffrement : aucune donnée sensible n'y est placée. JJWT vérifie la signature et l'expiration avant d'exposer les claims.

```
Authorization: Bearer <token>
sub = employe@sentinel.ma
role = EMPLOYEE
exp = création + 300 secondes
```

Vérification à chaque requête

JwtFilter relit l'utilisateur en base, vérifie enabled, son rôle et authVersion puis place une Authentication dans SecurityContextHolder. Le principal représente l'identifiant utilisateur. Une désactivation prend effet immédiatement, même avec un JWT encore valide. Le rôle du token ne constitue pas l'unique source de vérité.

Refresh token et déconnexion

L'access token dure cinq minutes. Le refresh token dure sept jours, possède une valeur aléatoire et est stocké en base sous forme de hash SHA-256. `POST /auth/refresh` révoque le jeton présenté et émet une nouvelle paire. Un verrou optimiste empêche la réutilisation concurrente normale. La déconnexion, le changement du mot de passe et de l'email incrémentent authVersion ; les anciens access et refresh tokens sont refusés.

Le frontend ne renouvelle pas automatiquement une session expirée : il respecte la redirection vers Login exigée au sujet. Un renouvellement explicite est proposé dans le profil tant que la session est active.

Configuration Spring Security

`SessionCreationPolicy.STATELESS`, routes `/auth/**` et `/h2-console/**` publiques, `/api/**` authentifiées, administration restreinte par rôle. CORS autorise les deux origines locales React. CSRF est désactivé pour l'API utilisant un header Bearer, sans authentification par cookie. La console H2 est limitée au développement local.

05. Autorisations et règles métier

Rôle et propriété

`@EnableMethodSecurity` active `@PreAuthorize`. `hasRole('ADMIN')` vérifie l'autorité `ROLE_ADMIN` ; `hasAuthority('ROLE_ADMIN')` permettrait la même vérification explicite. Les annotations protègent la couche service, y compris les appels hors HTTP. Les endpoints administratifs sont aussi protégés dans SecurityFilterChain.

Pour lire un incident, le service compare le principal authentifié à createdBy (employé) ou assignedTo (technicien). ADMIN peut consulter tous les incidents. Le technicien ne peut ajouter une note ou modifier un statut que dans son propre périmètre. L'employé ne dispose d'aucun endpoint lui permettant d'affecter un technicien ou de changer un statut.

Cycle de traitement

```
OPEN → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED
                                  ↑          |
                                  +----------+  reprise
```

Seul ADMIN affecte un technicien actif. Une affectation initiale transforme OPEN en ASSIGNED. Le traitement nécessite un technicien. RESOLVED peut être repris en IN_PROGRESS ou clôturé. CLOSED est terminal : changement de statut, réaffectation et ajout de notes sont refusés. Les mises à jour utilisent une transaction et le champ @Version pour détecter les conflits.

Historique et notes

Chaque événement comporte un identifiant, un auteur, un message, une date et son incident. L'affectation et le changement de statut sont enregistrés dans la même transaction que l'incident. Les notes sont séparées de l'historique ; leur ajout produit également un événement.

Validation et erreurs

Code	Signification et exemple
400	Données invalides, enum inconnu, titre vide, ancien mot de passe incorrect.
401	Authentification absente, jeton invalide ou expiré, compte désactivé.
403	Identité connue mais rôle ou propriété insuffisant.
404	Incident ou utilisateur introuvable.
409	Email existant, incident clôturé ou modification concurrente.

Les DTO utilisent @NotBlank, @Email, @Size, @NotNull et @Valid. @RestControllerAdvice centralise les réponses et les erreurs par champ. Les erreurs 401/403 issues des filtres sont également renvoyées en JSON.

06. Application React

AuthProvider conserve la session navigateur et fournit l'utilisateur aux composants. ProtectedRoute exige une session ; RoleRoute vérifie les rôles autorisés avant le rendu. Navbar adapte la navigation. Les services Axios regroupent les appels HTTP et ajoutent automatiquement le header Authorization.

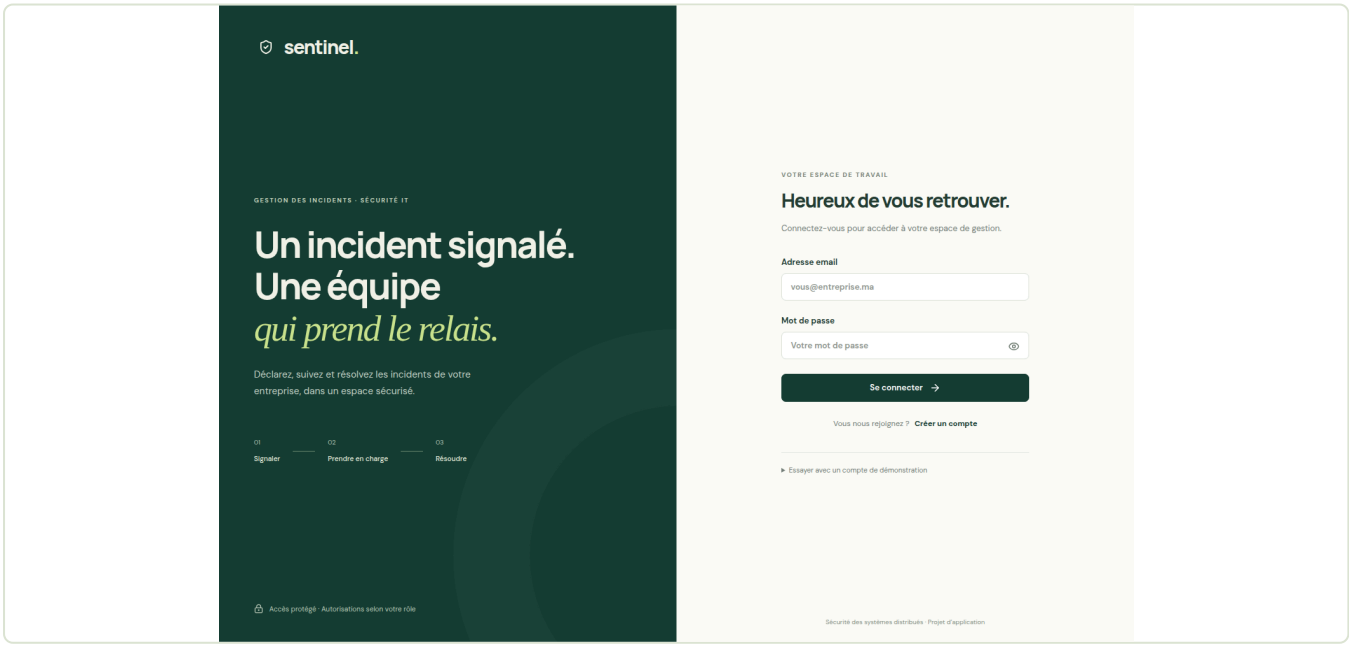
À la réception d'un 401, l'intercepteur supprime la session et émet un événement d'expiration. AuthProvider déclenche aussi une expiration locale à la date reçue du serveur. La page Login affiche un message explicite. La vraie vérification demeure celle du backend.

Page	Comportement
Login / Register	Connexion, inscription, validation et comptes de démonstration.
Dashboard	Statistiques calculées sur les données réelles autorisées et derniers incidents.
MyIncidents / AssignedIncidents / AdminIncidents	Listes par rôle, recherche et filtres statut/priorité.
NewIncident	Titre, description, catégorie et priorité.
IncidentDetails	Détails, historique, notes, affectation et transitions selon le rôle.
AdminUsers	Recherche, filtre de rôle, activation et désactivation avec confirmation.
Profile	Nom, email, ancien/nouveau mot de passe, confirmation, renouvellement de session.

Choix visuels

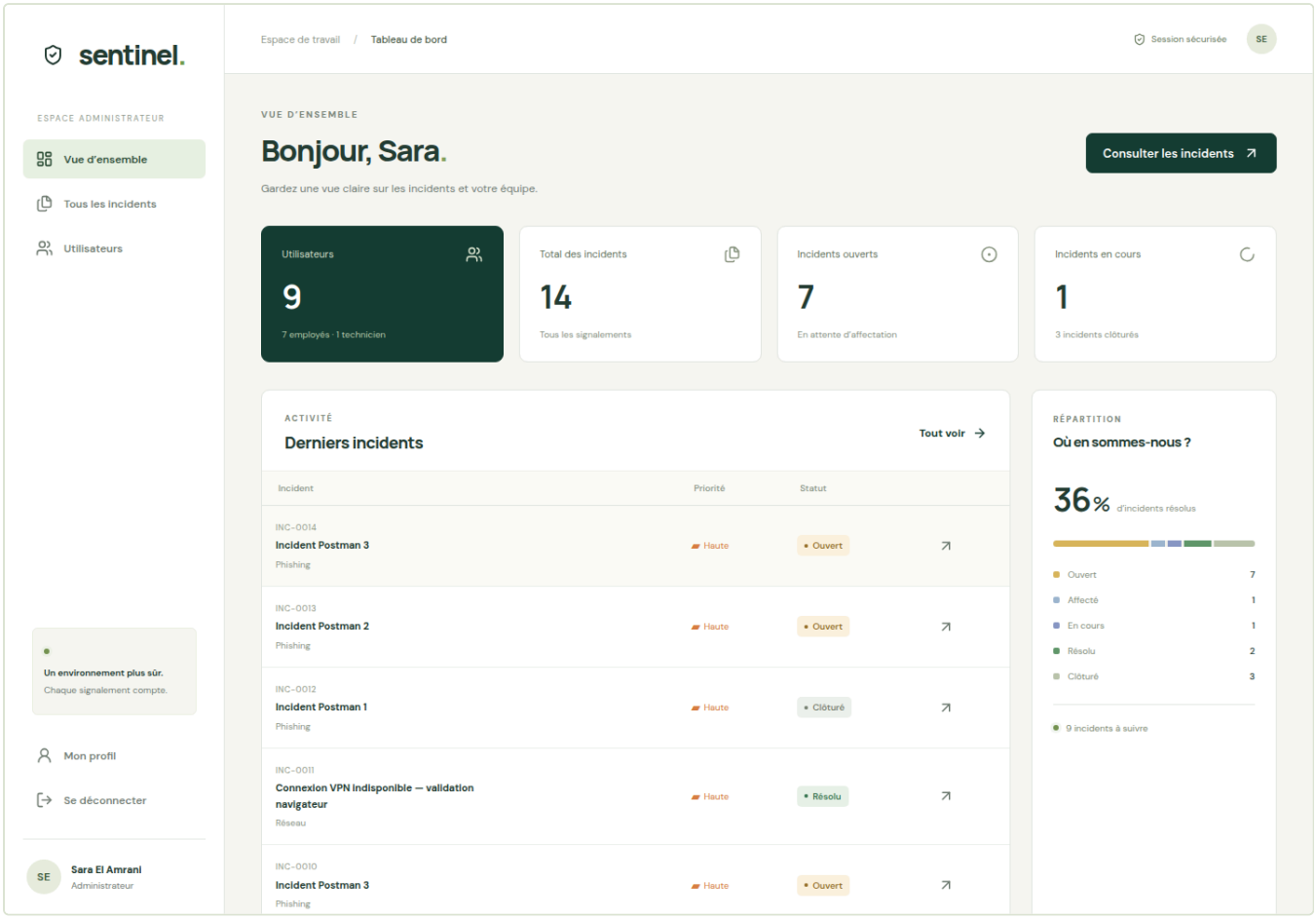
Palette vert forêt, fonds clairs, typographies DM Sans et Manrope, navigation latérale et tableaux lisibles. Les cartes résument les statistiques réellement calculées ; aucune donnée factice ne remplace une réponse API. Les formulaires proposent des états de chargement et d'erreur. Sur mobile, le menu devient escamotable, les colonnes se réorganisent et les tableaux restent défilables.

Capture de connexion



Capture réelle du navigateur pendant la vérification locale.

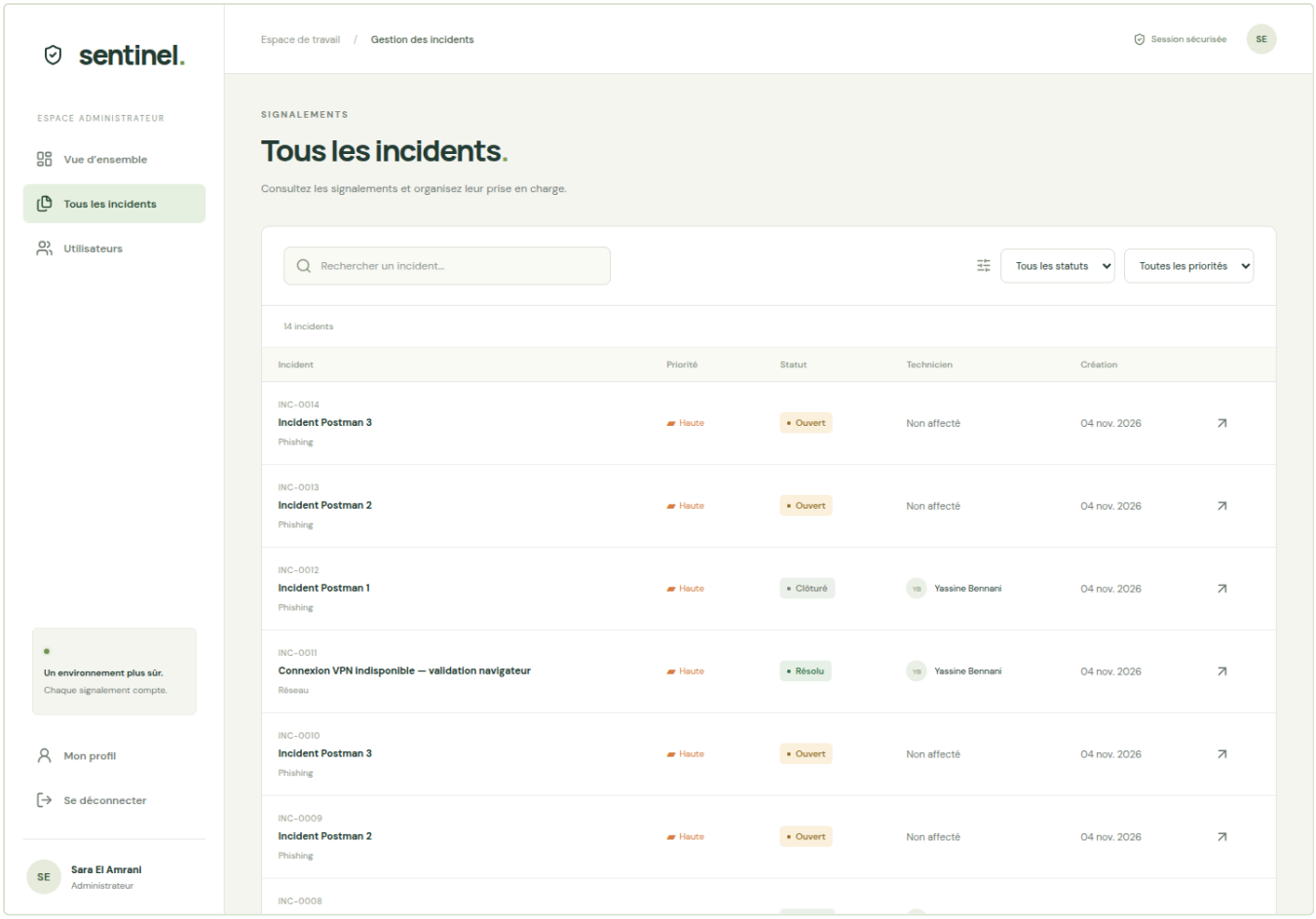
07.1. Administration — tableau de bord



Sara El Amrani : statistiques globales et derniers incidents.

Capture réelle du navigateur · 6 novembre 2026 · 1440 × 1000 pixels.

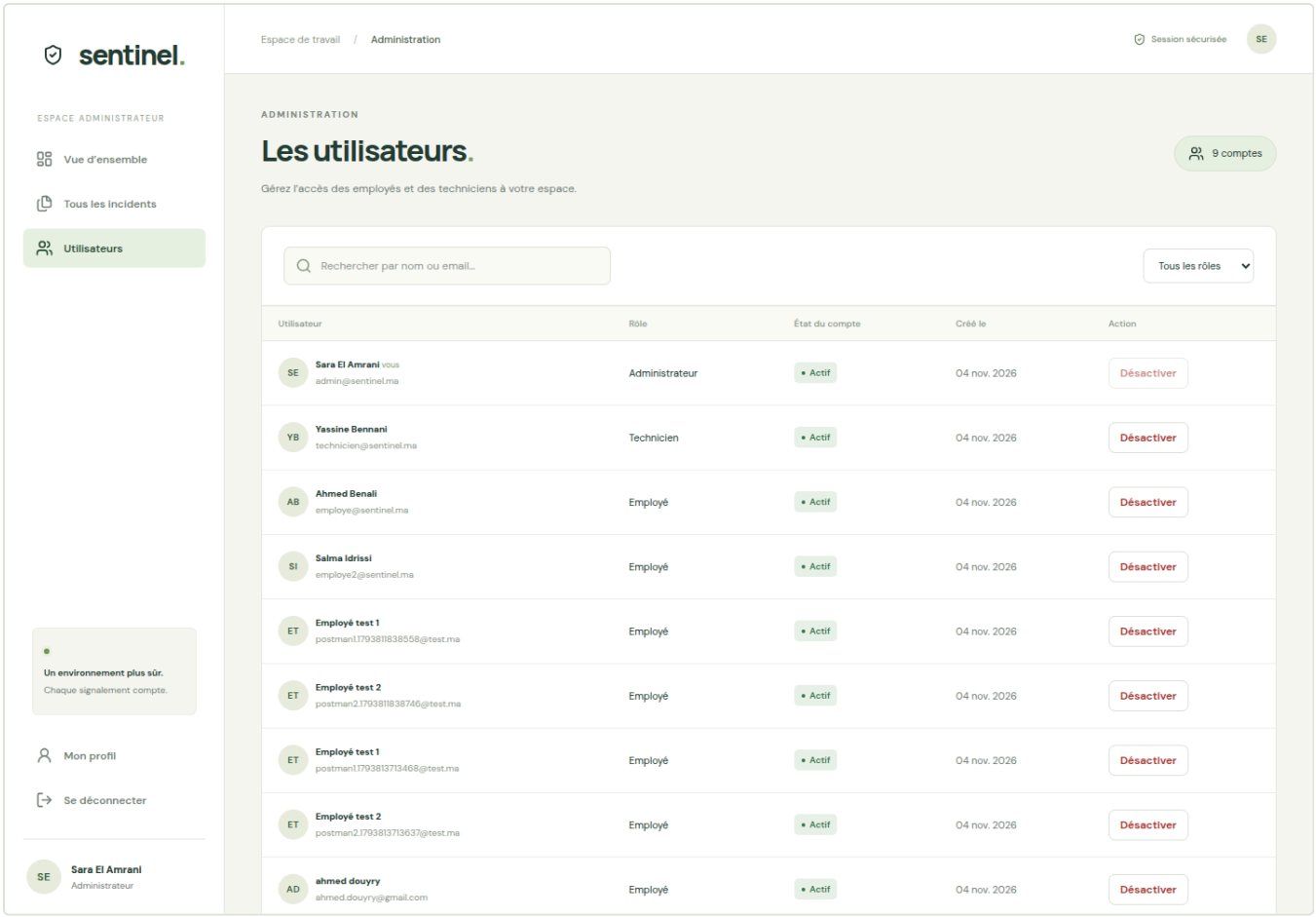
07.2. Administration — tous les incidents



Liste globale des signalements avec recherche et filtres de statut et de priorité.

Capture réelle du navigateur · 6 novembre 2026 · 1440 × 1000 pixels.

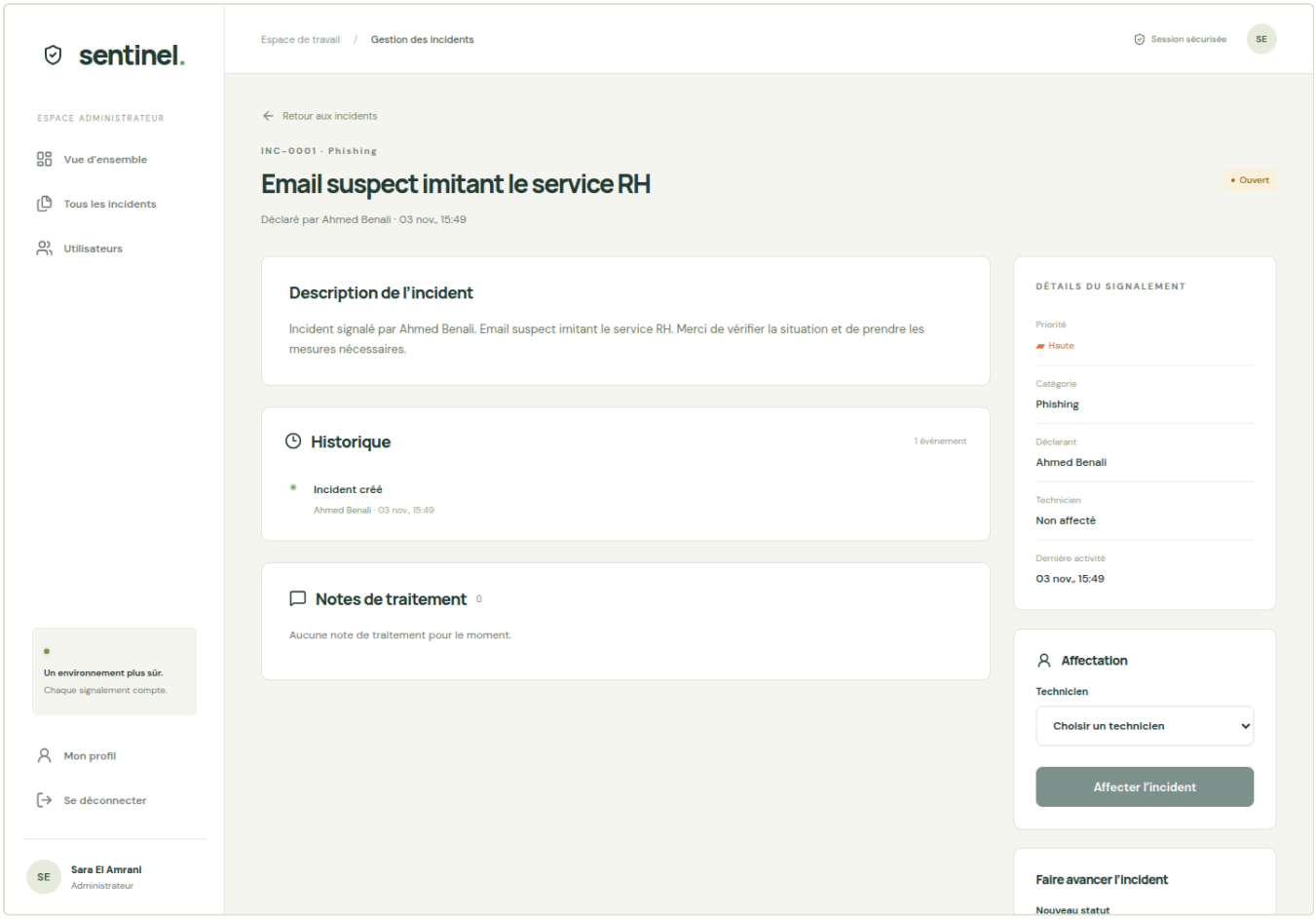
07.3. Administration — utilisateurs



Gestion des comptes, rôles et activation. L'administrateur ne peut pas désactiver son propre compte.

Capture réelle du navigateur · 6 novembre 2026 · 1440 × 1000 pixels.

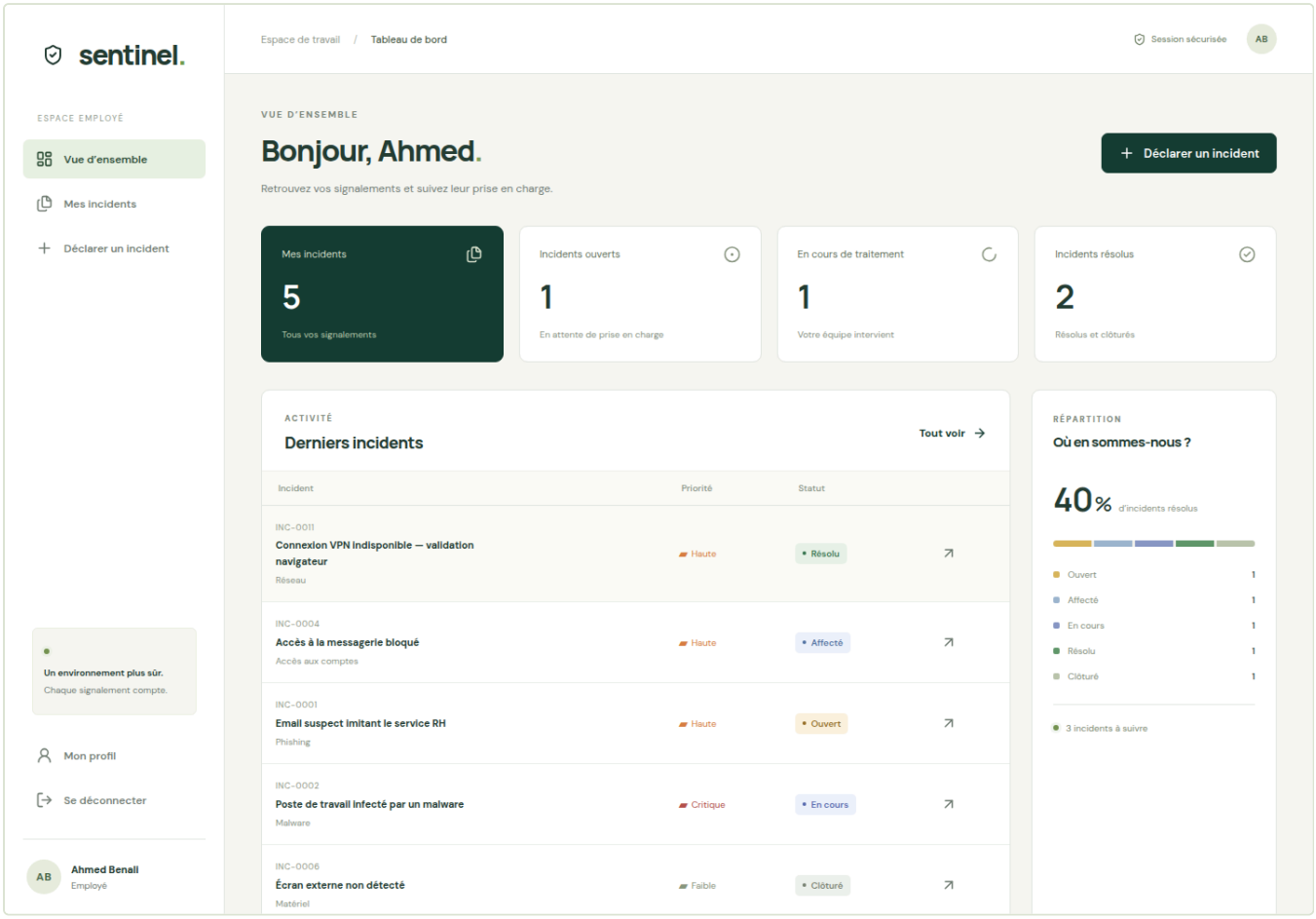
07.4. Administration — affectation



Incident déclaré par Ahmed Benali : détail, historique et formulaire d'affectation à un technicien actif.

Capture réelle du navigateur · 6 novembre 2026 · 1440 × 1000 pixels.

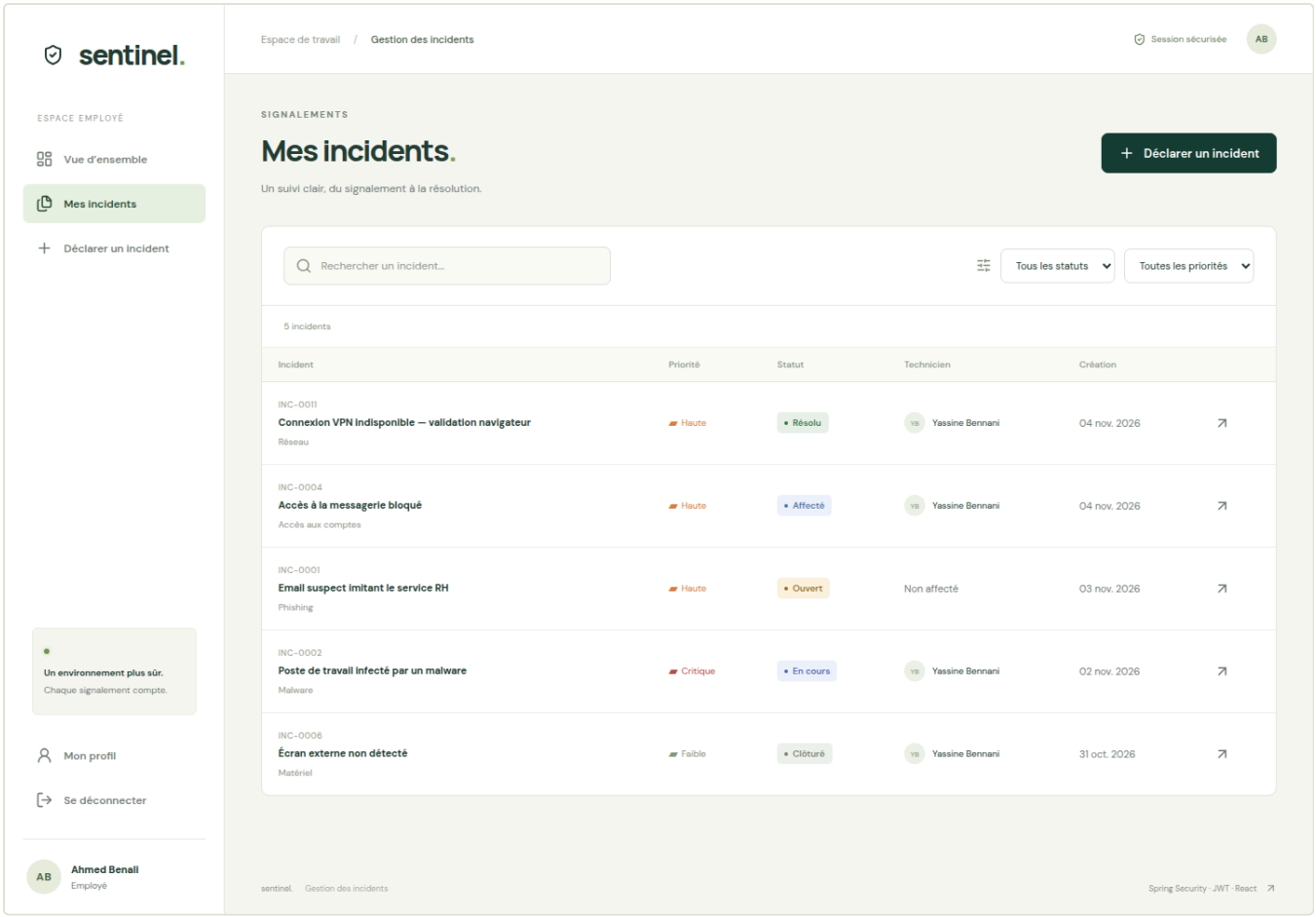
07.5. Employé — tableau de bord d’Ahmed



Ahmed Benali, connecté avec le compte employe@sentinel.ma : statistiques personnelles et derniers signalements.

Capture réelle du navigateur · 6 novembre 2026 · 1440 × 1000 pixels.

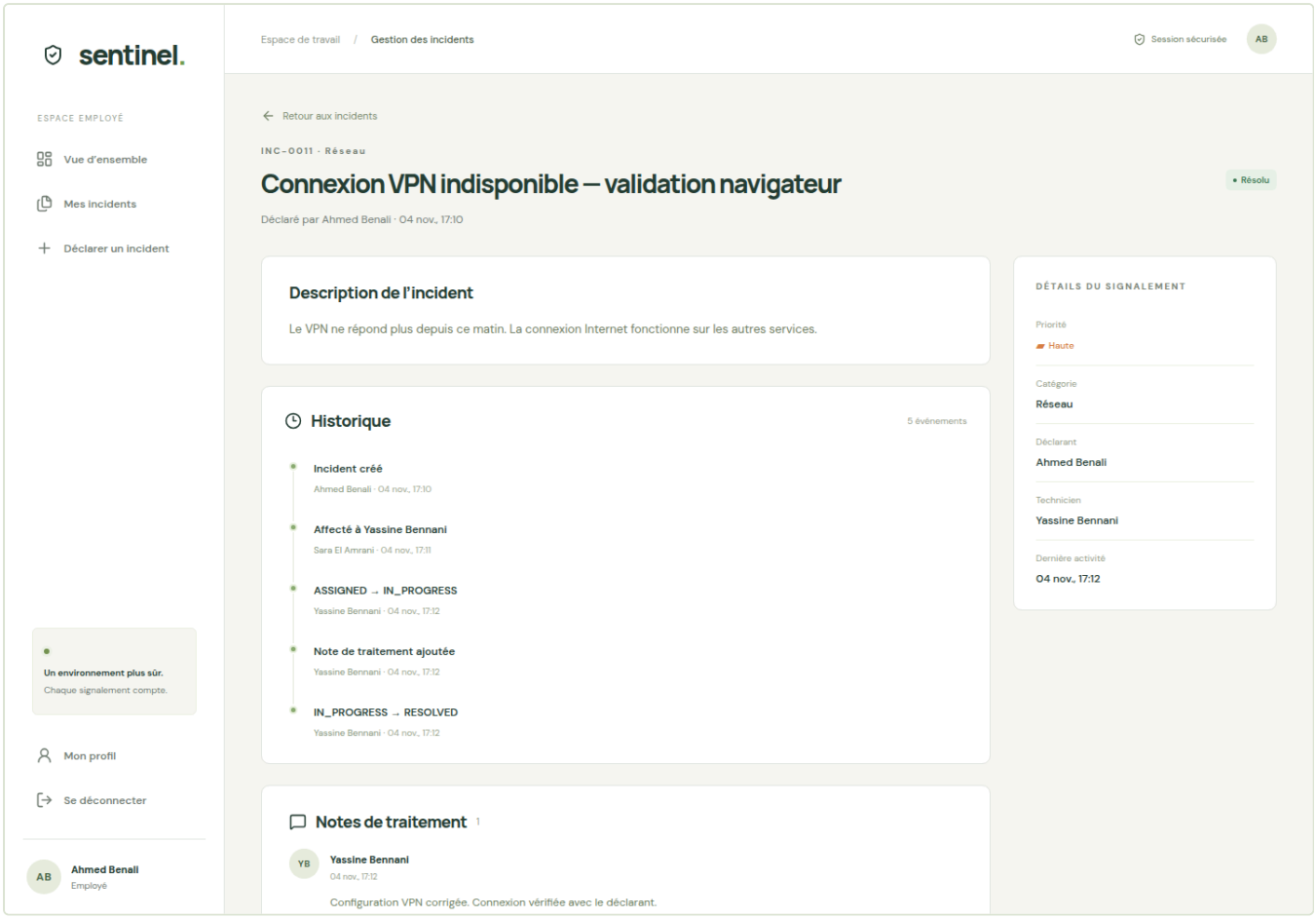
07.6. Employé — incidents d’Ahmed



Ahmed consulte ses cinq incidents, leurs priorités, statuts et techniciens affectés.

Capture réelle du navigateur · 6 novembre 2026 · 1440 × 1000 pixels.

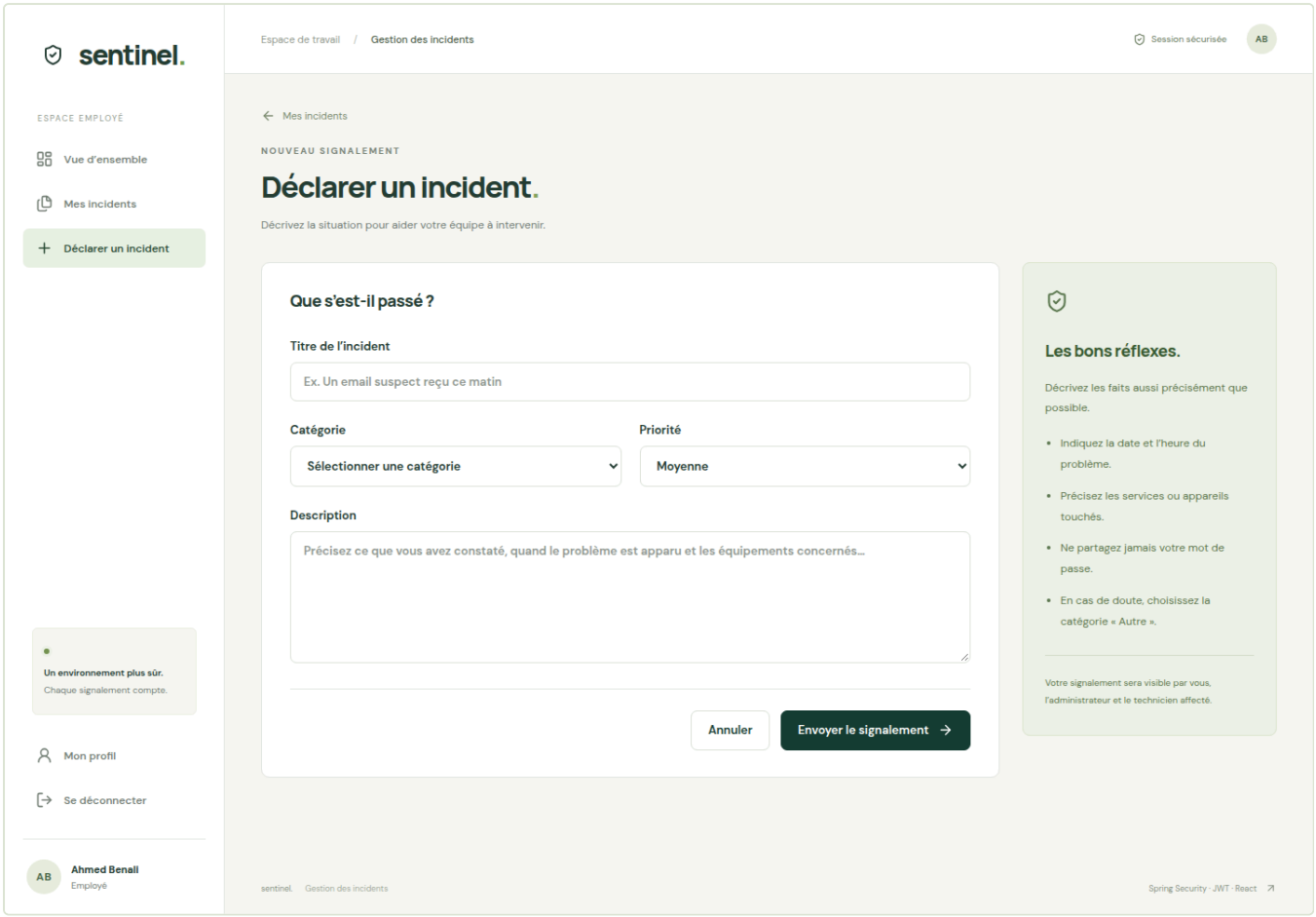
07.7. Employé — suivi et historique



Ahmed consulte son incident VPN résolu : affectation à Yassine Bennani, historique et note de traitement.

Capture réelle du navigateur · 6 novembre 2026 · 1440 × 1000 pixels.

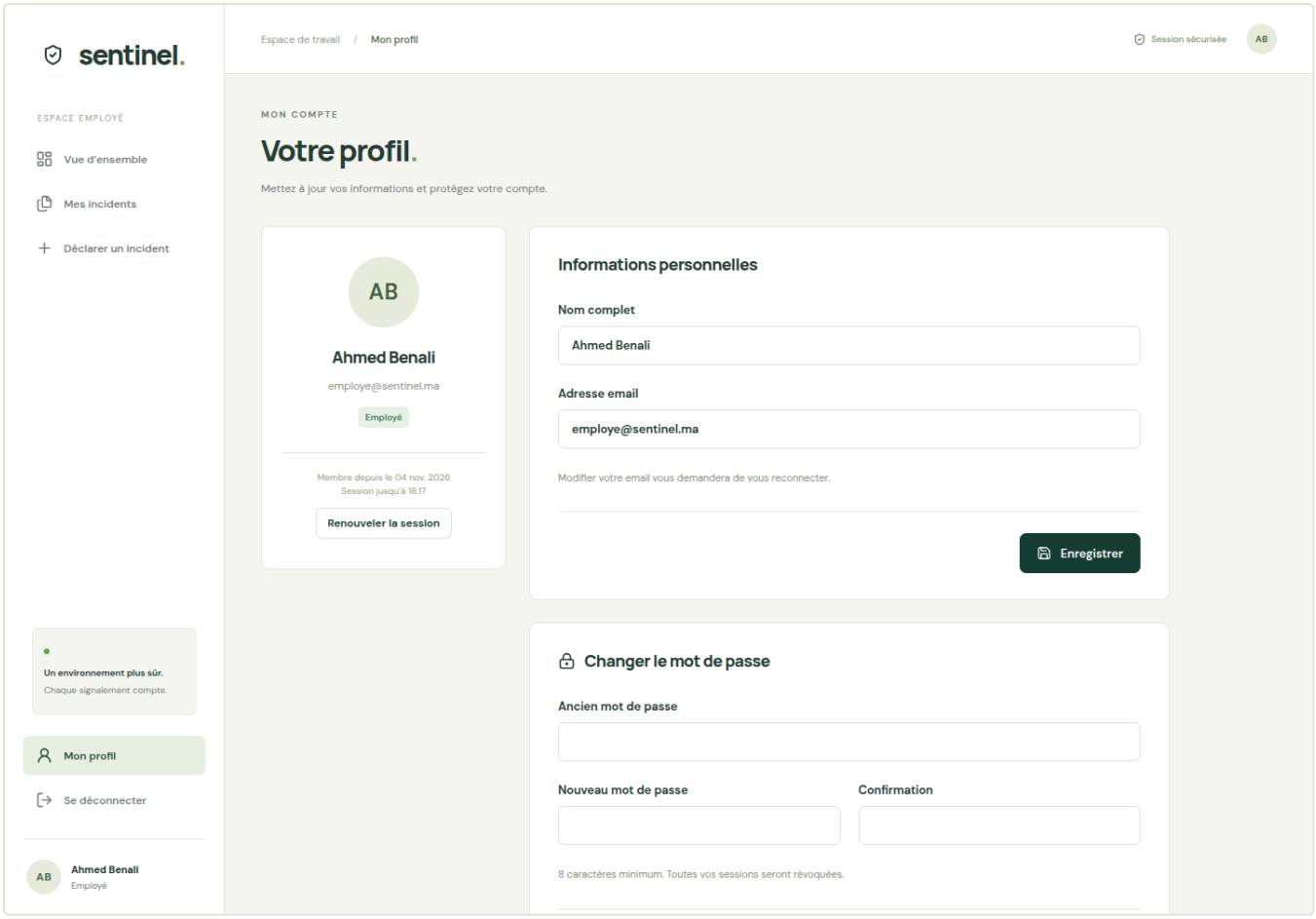
07.8. Employé — déclaration d’un incident



Formulaire de déclaration accessible à Ahmed : titre, description, catégorie et priorité. Aucune nouvelle déclaration n’a été soumise pour ces captures.

Capture réelle du navigateur · 6 novembre 2026 · 1440 × 1000 pixels.

07.9. Employé — profil d’Ahmed



Profil d’Ahmed Benali : informations personnelles, changement de mot de passe et renouvellement explicite de session.

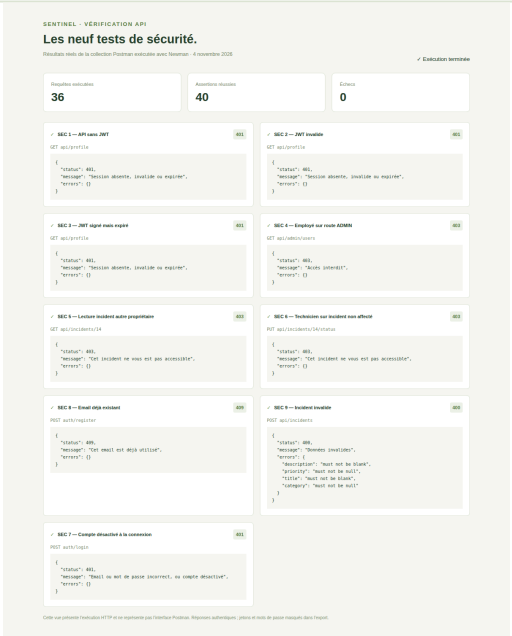
Capture réelle du navigateur · 6 novembre 2026 · 1440 × 1000 pixels.

08. Tests et preuve de fonctionnement

Les 25 tests Java ont réussi, sans échec ni erreur. Ils vérifient les règles dans une base H2 en mémoire indépendante. L’exécution Newman a validé les 36 requêtes et les 40 assertions, sans échec. La collection Postman comporte 36 requêtes et enchaîne inscriptions, déclarations, affectation, traitement, note, résolution, clôture et révocation des jetons.

Test de sécurité obligatoire	Résultat attendu
API protégée sans JWT	401
JWT invalide	401
JWT signé mais expiré	401
EMPLOYEE sur une route ADMIN	403
EMPLOYEE sur l’incident d’un autre	403
TECHNICIAN sur un incident non affecté	403
Compte désactivé à la connexion	401
Inscription avec email existant	409
Création avec données invalides	400

> Les résultats effectifs des commandes, les captures et les limites de vérification sont consignés dans docs/VALIDATION.md. Les preuves HTTP viennent de l’exécution de la collection par Newman. Pour les captures de l’interface Postman demandées dans le sujet, importer la collection et capturer son Runner lors de la présentation.



Vue des résultats HTTP. Cette image ne représente pas l’interface de Postman.

09. Lancement, démonstration et limites

Exécution locale

```
cd backend
mvn spring-boot:run

cd frontend
npm ci
npm run dev
```

Frontend : http://localhost:5173. Backend : http://localhost:8081. H2 : fichier local backend/data/sentinel. Les quatre comptes démo utilisent Demo2026!. Les identifiants complets et tous les endpoints sont documentés dans README.md.

Démonstration

Le guide docs/DEMONSTRATION.md reproduit le scénario du sujet : deux employés, trois incidents, refus d'accès croisé, affectation par ADMIN, traitement par le technicien, note, résolution et vérification de l'historique par l'employé.

Limites et améliorations

- Application de devoir local : secret et comptes de démonstration doivent être remplacés avant toute exposition publique.
- Stockage navigateur des jetons conforme au sujet ; une version de production demanderait un durcissement XSS/CSP et une réflexion sur les cookies HttpOnly.
- Pas de pagination ni de migrations de schéma versionnées ; H2 et ddl-auto=update suffisent pour la démonstration.
- Le déploiement MySQL n'est pas inclus : seul H2 est configuré et vérifié.
- Les captures d'interface Postman et la présentation orale restent à réaliser par l'étudiant avec la collection et le guide fournis.

Références de recherche

- Spring Security — Method Security (@PreAuthorize, @EnableMethodSecurity)
- Spring Security — Authorize HTTP Requests
- Jakarta Persistence — ManyToOne et FetchType
- Axios — Interceptors
- JJWT — Construction et vérification des claims
- RFC 7519 — JSON Web Token